
1、基础搭建.....	1
1.1 更改主机名.....	1
1.2 配置 hosts 文件.....	1
1.3 选择时区.....	1
1.4 配置时间服务器.....	2
1.5 制作定时任务.....	4
1.6 配置 ssh 免密.....	4
2、安装 JDK.....	4
3、安装 zookeeper.....	5
3.1 下载软件.....	5
3.2 配置 zookeeper 环境变量.....	5
3.3 配置文件 zoo.cfg.....	6
3.4 启动 ZooKeeper 集群.....	7
4、安装 hadoop.....	7
4.1 解压安装包，配置环境变量.....	7
4.2 配置 hadoop 各组件.....	8
5、安装 hive.....	12
5.1 slave2 上安装 MySQL server.....	12
5.2 数据仓库下载安装.....	14
5.3 hive 元数据配置到 MySQL(服务器端 slave1)	14
5.4 Master 作为客户端配置 hive.....	15
5.5 成功启动 Hive.....	16
6.安装 spark.....	17
6.1 安装 scala 环境.....	17
6.2 安装 Spark.....	17
8、数据统计分析.....	20
项目一：新冠疫情数据分析.....	20
9、重点说明.....	30

1、基础搭建

本次集群搭建共有三个节点，包括一个主节点 master，和两个从节点 slave1 和 slave2。具体操作如下：

1.1 更改主机名

1.使用本地连接工具 Xshell 或 MobaXterm 连接比赛平台所提供的 master, slave1, slave2 三台机器，并按照比赛平台提供的 linux 用户和密码进行登录，登录成功后开始进行接下来的比赛。

注意连接工具没有在桌面上，点击（或者键入）左下角“windows”，即可看到连接工具。

- 修改主机名：hostnamectl set-hostname master（在 master 执行）
- 立即生效：bash

```
[root@10-137-0-97 ~]# hostnamectl set-hostname master
[root@10-137-0-97 ~]# bash
[root@master ~]#
```

主机名已修改

同理修改 slave1 和 slave2 的主机名。

- hostnamectl set-hostname slave1（在 slave1 执行）
- hostnamectl set-hostname slave2（在 slave2 执行）

1.2 配置 hosts 文件

可以通过 ifconfig 命令进行查看机器的 ip 地址或者直接输入比赛平台提供的 ip 地址。

查看节点地址之后将三个节点的内网 ip 地址以及其对应的名称写进 hosts 文件。这里我们设置为 master、slave1、slave2。注意保存退出。

vim /etc/hosts（三台机器都执行）

```
[root@master ~]# vi /etc/hosts

127.0.0.1    localhost localhost.localdomain localhost4 localhost4.localdomain4
::1         localhost localhost.localdomain localhost6 localhost6.localdomain6

192.168.15.104 master
192.168.15.127 slave1
192.168.15.124 slave2
~
~
~
```

输入各节点相应IP

1.3 选择时区

- 调整时区：tzselect

```
[root@master ~]# tzselect 选择时区
Please identify a location so that time zone rules can be set correctly.
Please select a continent or ocean.
 1) Africa
 2) Americas
 3) Antarctica
 4) Arctic Ocean
 5) Asia ← 选择亚洲
 6) Atlantic Ocean
 7) Australia
 8) Europe
 9) Indian Ocean
10) Pacific Ocean
11) none - I want to specify the time zone using the Posix TZ format.
#? 5
Please select a country.
 1) Afghanistan      18) Israel           35) Palestine
 2) Armenia           19) Japan            36) Philippines
 3) Azerbaijan        20) Jordan           37) Qatar
 4) Bahrain           21) Kazakhstan       38) Russia
 5) Bangladesh        22) Korea (North)    39) Saudi Arabia
 6) Bhutan            23) Korea (South)    40) Singapore
 7) Brunei            24) Kuwait           41) Sri Lanka
 8) Cambodia          25) Kyrgyzstan       42) Syria
 9) China ← 中国          26) Laos            43) Taiwan
10) Cyprus            27) Lebanon          44) Tajikistan
11) East Timor        28) Macau            45) Thailand
12) Georgia           29) Malaysia         46) Turkmenistan
13) Hong Kong         30) Mongolia         47) United Arab Emirates
14) India             31) Myanmar (Burma)  48) Uzbekistan
15) Indonesia         32) Nepal            49) Vietnam
16) Iran              33) Oman             50) Yemen
17) Iraq              34) Pakistan

#? 9
Please select one of the following time zone regions.
 1) Beijing Time ← 北京时间
 2) Xinjiang Time
#? 1
The following information has been given:

      China
      Beijing Time

Therefore TZ='Asia/Shanghai' will be used.
Local time is now:   Fri Sep 28 20:33:01 CST 2018.
Universal Time is now: Fri Sep 28 12:33:01 UTC 2018.
Is the above information OK?
 1) Yes ← 覆盖时间
 2) No
#? 1
```

给出相应的提示之后，将时间设置写入环境变量。

```
echo "TZ='Asia/Shanghai'; export TZ" >> /etc/profile && source /etc/profile
```

1.4 配置时间服务器

环境中已经安装 ntp 服务，无需重复下载。可以使用命令进行查看：rpm -qa | grep ntp

1) master 作为 ntp 服务器：

修改 ntp 配置文件，屏蔽掉默认的 server，设置 master 作为局域网内时钟源（本地时间服务器），设置时间服务器的层级为 10，为集群主机提供时间服务。

```
vim /etc/ntp.conf
```

```
#注释掉 server 0 ~ n
```

```
#server 0.centos.pool.ntp.org iburst
#server 1.centos.pool.ntp.org iburst
#server 2.centos.pool.ntp.org iburst
#server 3.centos.pool.ntp.org iburst

server 127.127.1.0          # local clock
fudge 127.127.1.0 stratum 10  #stratum 设置为其它值也是可以的，其范围为 0~15
```

修改后注意保存退出。

```
# 注释
restrict
作用：对 ntp 做权限控制
ignore：忽略所有类型的 NTP 连接请求
nomodify：限制客户端不能使用命令 ntpc 和 ntpq 来修改服务器端的时间
noquery：不提供 NTP 网络校时服务
notrap：不接受远程登录请求
notrust：不接受没有经过认证的客户端的请求

【如果没有用任何参数，那么表示不做任何限制】
#开放本机的任何访问
restrict 127.0.0.1
restrict -6 ::1
restrict 10.220.5.0 mask 255.255.255.0 nomodify <<<允许 10.220.5.0/24 网段主机进行时间同步

server
作用：指定 ntp 服务器的地址
格式：server [ip or hostname] [prefer]
例子：server 127.127.1.0 <<<将当前主机作为时间服务器

fudge
作用：设置时间服务器的层级
格式：fudge ip [startnum int]
例子：fudge 10.225.5.1 startnum 10
注意：fudge 必须和 server 一块用，而且是在 server 的下一行
startnum
0~15
0：表示顶级
10：通常用于给局域网主机提供时间服务
```

2) master 上重启 ntp 服务

```
systemctl restart ntpd.service
```

3) 在 slave1, slave2 中进行同步：

```
ntpdate master
```

```
[root@slave2 ~]# ntpdate master
28 Sep 20:51:27 ntpdate[2338]: adjust time server 192.168.15.104 offset 0.201392 sec
[root@slave2 ~]#
```

同步master
时间

1.5 制作定时任务

环境中已经开启相应服务，查看 crontab 状态：service crond status

要求：从节点（slave1、slave2）在早十-晚五时间段每隔半个小时同步一次时间

crontab -e

输入 i

添加定时任务：*/30 10-17 * * * /usr/sbin/ntpdate master

查看定时任务列表：crontab -l

1.6 配置 ssh 免密

非对称机密算法--DSA

在 master 上执行如下命令生成公私密钥：（注意 master 上执行）

- ssh-keygen -t dsa -P "" -f ~/.ssh/id_dsa

然后将公钥文件内容添加到授权文件。

- cat /root/.ssh/id_dsa.pub >> /root/.ssh/authorized_keys #授权文件

可以将 master 授权文件复制到 slave1、slave2 信任授权设置。

- scp ~/.ssh/authorized_keys root@slave1:~/.ssh/
- scp ~/.ssh/authorized_keys root@slave2:~/.ssh/

非对称算法--RSA

在 master 上执行如下命令生成公私密钥：（注意 master 上执行，默认为 rsa 类型）

ssh-keygen

然后将公钥文件内容添加到授权文件。

ssh-copy-id -i /root/.ssh/id_rsa.pub master （或者使用 localhost 也可已完成本地免密）

可以将 master 授权文件复制到 slave1、slave2 信任授权设置。

ssh-copy-id -i /root/.ssh/id_rsa.pub slave1

ssh-copy-id -i /root/.ssh/id_rsa.pub slave2

注意：SSH 只需要 namenode 和 datanode 之间通信，datanode 之间并没有交互，所以 SSH 公钥只需要 namenode 和 datanode 之间交换。

2、安装 JDK

以下操作为先在 master 上操作，然后远程复制到 slave1 和 slave2。参赛选手仅供参考。

首先在根目录下建立工作路径/usr/java。

```
mkdir -p /usr/java
```

进入创建的 java 工作路径。

```
cd /usr/java
```

解压 java 安装包(master 上执行)。

```
tar -zxvf /usr/package277/jdk-8u221-linux-x64.tar.gz -C /usr/java/
```

- 修改环境变量: vim /etc/profile

添加内容如下:

```
export JAVA_HOME=/usr/java/jdk1.8.0_221
export CLASSPATH=$JAVA_HOME/lib/
export PATH=$PATH:$JAVA_HOME/bin
export PATH JAVA_HOME CLASSPATH
```

- 生效环境变量: source /etc/profile

- 查看 java 版本: java -version

```
[root@slave2 java]# java -version
java version "1.8.0_221"
Java(TM) SE Runtime Environment (build 1.8.0_221-b11)
Java HotSpot(TM) 64-Bit Server VM (build 25.221-b11, mixed mode)
```

在 master 中将 JDK 复制到 slave1 和 slave2 中。(在 master 执行)

```
scp -r /usr/java root@slave1:/usr/
```

```
scp -r /usr/java root@slave2:/usr/
```

注意: 此时需要去 slave1 和 slave2 上配置 java 的环境变量, 并使环境变量生效。

3、安装 zookeeper

3.1 下载软件

创建工作路径/usr/zookeeper, 解压软件至工作路径。

```
mkdir -p /usr/zookeeper && cd /usr/zookeeper
```

```
tar -zxvf /usr/package277/zookeeper-3.4.14.tar.gz -C /usr/zookeeper/
```

3.2 配置 zookeeper 环境变量

```
vim /etc/profile
```

```
#zookeeper
```

```
export ZOOKEEPER_HOME=/usr/zookeeper/zookeeper-3.4.14
PATH=$PATH:$ZOOKEEPER_HOME/bin
```

生效环境变量：source /etc/profile

3.3 配置文件 zoo.cfg

1.进入 zookeeper 配置文件夹 conf，将 zoo_sample.cfg 文件拷贝一份命名为 zoo.cfg，Zookeeper 在启动时会找这个文件作为默认配置文件。

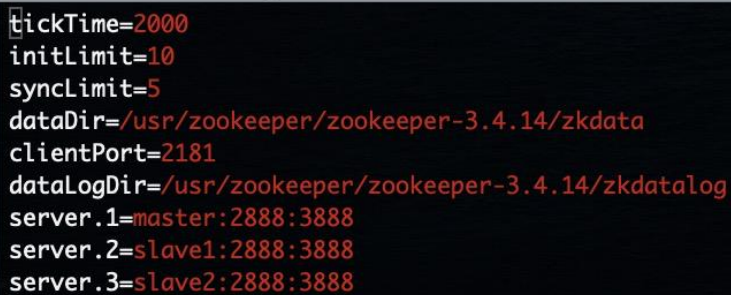
```
cd /usr/zookeeper/zookeeper-3.4.14/conf/ && vim zoo.cfg
```

2.对 zoo.cfg 文件配置如下：

```
vim zoo.cfg
```

修改如下：

```
tickTime=2000
initLimit=10
syncLimit=5
dataDir=/usr/zookeeper/zookeeper-3.4.14/zkdata
clientPort=2181
dataLogDir=/usr/zookeeper/zookeeper-3.4.14/zkdataLog
server.1=master:2888:3888
server.2=slave1:2888:3888
server.3=slave2:2888:3888
```



```
tickTime=2000
initLimit=10
syncLimit=5
dataDir=/usr/zookeeper/zookeeper-3.4.14/zkdata
clientPort=2181
dataLogDir=/usr/zookeeper/zookeeper-3.4.14/zkdataLog
server.1=master:2888:3888
server.2=slave1:2888:3888
server.3=slave2:2888:3888
```

3.在 zookeeper 的目录中，创建配置中所需的 zkdata 和 zkdataLog 两个文件夹。（在 master 执行）

```
cd /usr/zookeeper/zookeeper-3.4.14 && mkdir zkdata zkdataLog
```

在 master 中将 zookeeper 复制到 slave1 和 slave2 中。（在 master 执行）

```
scp -r /usr/zookeeper root@slave1:/usr/
```

```
scp -r /usr/zookeeper root@slave2:/usr/
```

注意修改 myid 和环境变量。

4.进入 zkdata 文件夹，创建文件 myid，用于表示是几号服务器，用来标识当前主机。

master 主机中，设置服务器 id 为 1。（集群中设置 master 为 1 号服务器，slave1 为 2 号服务器，slave2 为 3 号服务器）

```
cd /usr/zookeeper/zookeeper-3.4.14/zkdata
```

```
echo 1 >> myid
```

3.4 启动 ZooKeeper 集群

在 ZooKeeper 集群的每个结点上，执行启动 ZooKeeper 服务的脚本。注意在 zookeeper 目录下：（三台机器都执行）

开启服务：zkServer.sh start

查看状态：zkServer.sh status(稍等片刻)

```
[root@master zookeeper-3.4.10]# cd zkdata
[root@master zkdata]# echo 1 >> myid
[root@master zkdata]# cat myid
1
[root@master zkdata]# zkServer.sh start
ZooKeeper JMX enabled by default
Using config: /usr/zookeeper/zookeeper-3.4.10/bin/conf/zoo.cfg
Starting zookeeper ... STARTED
[root@master zkdata]# zkServer.sh status
ZooKeeper JMX enabled by default
Using config: /usr/zookeeper/zookeeper-3.4.10/bin/conf/zoo.cfg
Mode: follower
[root@master zkdata]#

[root@slave1 zookeeper-3.4.10]# cd zkdata
[root@slave1 zkdata]# echo 2 >> myid
[root@slave1 zkdata]# cat myid
2
[root@slave1 zkdata]# zkServer.sh start
ZooKeeper JMX enabled by default
Using config: /usr/zookeeper/zookeeper-3.4.10/bin/conf/zoo.cfg
Starting zookeeper ... STARTED
[root@slave1 zkdata]# zkServer.sh status
ZooKeeper JMX enabled by default
Using config: /usr/zookeeper/zookeeper-3.4.10/bin/conf/zoo.cfg
Mode: follower
[root@slave1 zkdata]#

[root@slave2 zookeeper-3.4.10]# cd zkdata
[root@slave2 zkdata]# echo 3 >> myid
[root@slave2 zkdata]# cat myid
3
[root@slave2 zkdata]# zkServer.sh start
ZooKeeper JMX enabled by default
Using config: /usr/zookeeper/zookeeper-3.4.10/bin/conf/zoo.cfg
Starting zookeeper ... STARTED
[root@slave2 zkdata]# zkServer.sh status
ZooKeeper JMX enabled by default
Using config: /usr/zookeeper/zookeeper-3.4.10/bin/conf/zoo.cfg
Mode: leader
[root@slave2 zkdata]#

[root@slave1 zookeeper-3.4.10]# cd zkdata
[root@slave1 zkdata]# echo 2 >> myid
[root@slave1 zkdata]# cat myid
2
[root@slave1 zkdata]# zkServer.sh start
ZooKeeper JMX enabled by default
Using config: /usr/zookeeper/zookeeper-3.4.10/bin/conf/zoo.cfg
Starting zookeeper ... STARTED
[root@slave1 zkdata]# zkServer.sh status
ZooKeeper JMX enabled by default
Using config: /usr/zookeeper/zookeeper-3.4.10/bin/conf/zoo.cfg
Mode: follower
[root@slave1 zkdata]#

[root@slave2 zookeeper-3.4.10]# cd zkdata
[root@slave2 zkdata]# echo 3 >> myid
[root@slave2 zkdata]# cat myid
3
[root@slave2 zkdata]# zkServer.sh start
ZooKeeper JMX enabled by default
Using config: /usr/zookeeper/zookeeper-3.4.10/bin/conf/zoo.cfg
Starting zookeeper ... STARTED
[root@slave2 zkdata]# zkServer.sh status
ZooKeeper JMX enabled by default
Using config: /usr/zookeeper/zookeeper-3.4.10/bin/conf/zoo.cfg
Mode: leader
[root@slave2 zkdata]#
```

通过上面状态查询结果可见，一个节点是 Leader，其余的节点是 Follower。

至此，zookeeper 安装成功。

4、安装 hadoop

以下步骤逻辑仅供选手参考：即 hadoop 安装同样在主节点 master 上进行配置操作，然后将文件复制到子节点 slave1 和 slave2。

4.1 解压安装包，配置环境变量

1. 创建工作路径/usr/hadoop，相应软件解压至工作路径。

```
mkdir -p /usr/hadoop && cd /usr/hadoop
```

```
tar -zxvf /usr/package277/hadoop-2.7.7.tar.gz -C /usr/hadoop/
```

2.添加 hadoop 系统变量（三台机器执行）

vim /etc/profile

添加如下内容：

```
#hadoop
#配置 Hadoop 安装目录
export HADOOP_HOME=/usr/hadoop/hadoop-2.7.7
#在原 PATH 基础上加入 Hadoop 的 bin 和 sbin 目录
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
```

使用以下命令使 profile 生效：

source /etc/profile

4.2 配置 hadoop 各组件

hadoop 的各个组件的都是使用 XML 进行配置，这些文件存放在 hadoop 的 etc/hadoop 目录下。

配置文件	配置对象	主要内容
hadoop-env.sh	hadoop 运行环境	用来定义 Hadoop 运行环境相关的配置信息；
core-site.xml	集群全局参数	定义系统级别的参数，包括 HDFS URL、Hadoop 临时目录等；
hdfs-site.xml	HDFS 参数	定义名称节点、数据节点的存放位置、文本副本的个数、文件读取权限等；
mapred-site.xml	MapReduce 参数	包括 JobHistory Server 和应用程序参数两部分，如 reduce 任务的默认个数、任务所能够使用内存的默认上下限等；
yarn-site.xml	集群资源管理系统参数	配置 ResourceManager，nodeManager 的通信端口，web 监控端口等；

1. hadoop-env.sh 运行环境

cd \$HADOOP_HOME/etc/hadoop

vim hadoop-env.sh

输入以下内容，修改 java 环境变量：

```
export JAVA_HOME=/usr/java/jdk1.8.0_221
```

也可以直接输入命令：echo "export JAVA_HOME=/usr/java/jdk1.8.0_221" >> hadoop-env.sh

键入“Esc”，退出编辑模式，使用命令“:wq”进行保存退出。

2. core-site.xml 全局参数

vim core-site.xml

```
<configuration>
<property>
```

```

<name>fs.default.name</name>
<!-- 用于指定 NameNode 的地址 -->
<value>hdfs://master:9000</value>
</property>
<property>
  <name>hadoop.tmp.dir</name>
  <!-- Hadoop 运行时产生文件的临时存储目录 -->
  <value>/root/hadoopData/tmp</value>
</property>
</configuration>

```

master: 在主节点的 ip 或者映射名。

9000: 主节点和从节点配置的端口都是 9000。

3. hdfs-site.xml 配置文件

dfs.replication: 因为 hadoop 是具有可靠性的, 它会备份多个文本, 这里 value 就是指备份的数量 (小于等于从节点的数量)。

vim hdfs-site.xml

在<configuration></configuration>中加入以下代码:

```

<property>
  <name>dfs.replication</name>
  <!-- 数据块副本的数量, 默认为 3 -->
  <value>2</value>
</property>
<property>
  <name>dfs.namenode.name.dir</name>
  <!-- NameNode 在本地文件系统中持久存储命名空间和事务日志的路径 -->
  <value>/root/hadoopData/name</value>
</property>
<property>
  <name>dfs.datanode.data.dir</name>
  <!-- DataNode 在本地文件系统中存放块的路径 -->
  <value>/root/hadoopData/data</value>
</property>
<property>
  <name>dfs.permissions</name>
  <!-- 关闭 hadoop 集群权限校验 -->
  <value>false</value>
</property>
<property>
  <name>dfs.datanode.use.datanode.hostname</name>
  <!-- datanode 之间的通信也通过域名方式 -->
  <value>true</value>
</property>

```

4. YARN 框架运行环境

找到 JAVA_HOME 参数位置，将前面的#去掉，将其值修改为本机安装的 JDK 的实际位置。

vim yarn-env.sh

```
export JAVA_HOME=/usr/java/jdk1.8.0_221
```

也可以直接输入命令：

```
echo "export JAVA_HOME=/usr/java/jdk1.8.0_221" >> yarn-env.sh
```

5. YARN 框架的核心配置

YARN 框架的核心配置文件，用于配置 YARN 进程及 YARN 的相关属性。

首先需要指定 ResourceManager 守护进程所在主机，默认为 0.0.0.0，即当前设备，所以这里指定为 master，端口为 18141；

其次需要设置 NodeManager 上运行的辅助服务，需配置成 mapreduce_shuffle 才可运行 MapReduce 程序。

vim yarn-site.xml

在<configuration></configuration>中加入以下代码：

```
<property>
  <name>yarn.resourcemanager.admin.address</name>
  <value>master:18141</value>
</property>
<property>
  <name>yarn.nodemanager.aux-services</name>
  <!-- 指定 reducer 获取数据的方式-->
  <!-- NodeManager 上运行的附属服务，也可以理解为 mapreduce 获取数据的方式 -->
  <value>mapreduce_shuffle</value>
</property>
<property>
  <name>yarn.nodemanager.auxservices.mapreduce.shuffle.class</name>
  <value>org.apache.hadoop.mapred.ShuffleHandler</value>
</property>
```

6. 编辑配置计算框架

该文件是 MapReduce 的核心配置文件，用于指定 MapReduce 运行时框架。

cp mapred-site.xml.template mapred-site.xml && vim mapred-site.xml

```
[root@master hadoop]# cp mapred-site.xml.template mapred-site.xml
[root@master hadoop]# vim mapred-site.xml
[root@master hadoop]#
```

在<configuration></configuration>中加入以下代码：

```
<property>
  <!--指定 Mapreduce 运行在 yarn 上-->
  <!-- 指定使用 YARN 运行 MapReduce 程序，默认为 local -->
```

```
<name>mapreduce.framework.name</name>
<value>yarn</value>
</property>
```

注意保存退出。

7. 编写 slaves 和 master 文件

编写 slaves 文件，添加子节点 slave1 和 slave2；

编写 master 文件，添加主节点 master。

```
echo master > master && echo slave1 > slaves && echo slave2 >> slaves
```

在 master 中将 hadoop 复制到 slave1 和 slave2 中。（在 master 执行）

```
scp -r /usr/hadoop root@slave1:/usr/
```

```
scp -r /usr/hadoop root@slave2:/usr/
```

8. 格式化 hadoop

- 格式化 namenode: `hadoop namenode -format` (仅在 master 中进行操作)

当出现 “Storage directory /usr/hadoop/hadoop-2.7.7/hdfs/name has been successfully formatted.” 的时候，表明格式化成功。

#格式化本质是进行文件系统的初始化操作，创建一些 hadoop 自己所需要的文件。

9. 开启 hadoop 集群

仅在 master 主机上开启操作命令。它会带起从节点的启动。（仅在 master 中进行操作）

- 主节点开启服务: `start-all.sh`

The image contains four terminal screenshots arranged in a 2x2 grid, showing the startup of Hadoop services. The top row shows the output of the `jps` command on the master and slave nodes. The bottom row shows the output of the `cat VERSION` command on the master and slave nodes.

Top-left (Master):

```
[root@master hdfs]# jps
11248 NameNode
11441 SecondaryNameNode
11601 ResourceManager
11863 Jps
10158 QuorumPeerMain
[root@master hdfs]#
```

Top-right (Slave1):

```
[root@slave1 hadoop]# vim
[root@slave1 hadoop]# jps
10018 QuorumPeerMain
10983 DataNode
11195 Jps
11087 NodeManager
[root@slave1 hadoop]#
```

Top-right (Slave2):

```
[root@slave2 hadoop]# vim
[root@slave2 hadoop]# jps
11170 Jps
11062 NodeManager
10030 QuorumPeerMain
10958 DataNode
[root@slave2 hadoop]#
```

Bottom-left (Master):

```
[root@master current]# cat VERSION
#Thu Jun 18 13:12:47 CST 2020
namespaceID=235644377
clusterID=CID-3d790bd7-d635-48ac-b861-6a5915518a38
ctime=0
storageType=NAME_NODE
blockpoolID=BP-790213631-172.16.44.95-1592457167778
layoutVersion=-63
[root@master current]#
```

Bottom-right (Slave1):

```
[root@slave1 current]# cat VERSION
#Thu Jun 18 05:19:48 UTC 2020
storageID=DS-fae7eb4e-6151-4414-92f3-2b27885902a5
clusterID=CID-3d790bd7-d635-48ac-b861-6a5915518a38
ctime=0
datanodeUuid=cbe6f36f-6499-477a-800a-c821eea5490d
storageType=DATA_NODE
layoutVersion=-56
[root@slave1 current]#
```

使用 hadoop 命令 “`hadoop fs`” 进行相关操作。

查看集群状态: `hdfs dfsadmin -report`

```
[root@master hadoop]# hdfs dfsadmin -report
Configured Capacity: 85865299968 (79.97 GB)
Present Capacity: 63886135296 (59.50 GB)
DFS Remaining: 63886127104 (59.50 GB)
DFS Used: 8192 (8 KB)
DFS Used%: 0.00%
Under replicated blocks: 0
Blocks with corrupt replicas: 0
Missing blocks: 0
Missing blocks (with replication factor 1): 0
```

Live datanodes (2):

```
Name: 172.18.32.27:50010 (slave1)
Hostname: slave1
Decommission Status : Normal
Configured Capacity: 42932649984 (39.98 GB)
DFS Used: 4096 (4 KB)
Non DFS Used: 10989735936 (10.23 GB)
DFS Remaining: 31942909952 (29.75 GB)
DFS Used%: 0.00%
DFS Remaining%: 74.40%
Configured Cache Capacity: 0 (0 B)
Cache Used: 0 (0 B)
Cache Remaining: 0 (0 B)
Cache Used%: 100.00%
Cache Remaining%: 0.00%
Xceivers: 1
Last contact: Sun Sep 26 01:56:11 CST 2021
```

5、安装 hive

实验中我们选用 hive 的远程模式, slave2 安装 mysql server 用于存放元数据, slave1 作为 hive server 作为 thrift 服务器, master 作为 client 客户端进行操作。

5.1 slave2 上安装 MySQL server

1.配置过本地源了, 环境中已经安装 MySQL Server, 同时环境中已经关闭开机自启服务, 本步骤无需操作。

- 关闭开机自启: **systemctl disable mysqld**
- 查看开启自启服务: **systemctl list-unit-files**
- 查看是否加入开机启动项: **systemctl list-unit-files |grep mysqld.service**

```
[root@iZ8vb537hyrlcgo36jnn7yZ yum.repos.d]# systemctl list-unit-files |grep mysqld.service
mysqld.service                                _ disabled
```

2.启动服务

- 开启服务: **systemctl start mysqld**

第一次启动后会有个初始化的过程, 会产生 root 账户的随机密码

3.登陆 MySQL

安装完毕后, MySQL 会在 /var/log/mysqld.log 这个文件中会自动生成一个随机的密码, 获取这个随机密码, 以用于登录 MySQL 数据库:

- 获取初密码: **grep "temporary password" /var/log/mysqld.log**
- 登陆 MySQL: **mysql -uroot -p** (注意中英文)

```

[root@slave2 ~]# grep "temporary password" /var/log/mysql.log 获取初始密码
2018-10-09T03:20:29.065471Z 1 [Note] A temporary password is generated for root@localhost: A8Q&i!2G,zf<
[root@slave2 ~]# mysql -uroot -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.  登陆MySQL
Your MySQL connection id is 2
Server version: 5.7.23

Copyright (c) 2000, 2018, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>

```

4. MySQL 密码安全策略设置

设置密码强度为低级: `set global validate_password_policy=0;`

设置密码长度: `set global validate_password_length=4;`

修改本地密码: `alter user 'root'@'localhost' identified by '123456';`

密码强度分级如下:

- 0 为 low 级别, 只检查长度;
- 1 为 medium 级别 (默认), 符合长度为 8, 且必须含有数字, 大小写, 特殊字符;
- 2 为 strong 级别, 密码难度更大一些, 需要包括字典文件。
- 密码长度最低长为 4, 当设置长度为 1、2、3 时, 其长度依然为 4。

5. 设置远程登录

- 创建用户: `create user 'root'@'%' identified by '123456';`
- 允许远程连接: `grant all privileges on *.* to 'root'@'%' with grant option;`
- 刷新权限: `flush privileges;`

同等命令: `GRANT ALL PRIVILEGES ON *.* TO 'root'@'%' IDENTIFIED BY '123456' WITH GRANT OPTION;`

#将 host 字段的值改为%就表示在任何客户端机器上能以 root 用户登录到 MySQL 服务器, 建议在开发时设为%。

```

mysql> create user 'root'@'%' identified by '123456';
Query OK, 0 rows affected (0.00 sec)

mysql> grant all privileges on *.* to 'root'@'%' with grant option;
Query OK, 0 rows affected (0.00 sec)

mysql> flush privileges;
Query OK, 0 rows affected (0.01 sec)

mysql>

```

5.2 数据仓库下载安装

1. 下载解压

同样 master 上建立文件夹/usr/hive，然后 master 中将安装包远程复制到 slave1。

```
mkdir -p /usr/hive && cd /usr/hive
```

```
tar -zxvf /usr/package277/apache-hive-2.3.4-bin.tar.gz -C /usr/hive/
```

2. 设置系统变量

修改/etc/profile 文件设置 hive 环境变量。（master 和 slave1 都执行）

- vim /etc/profile

```
#hive
export HIVE_HOME=/usr/hive/apache-hive-2.3.4-bin
export PATH=$PATH:$HIVE_HOME/bin
```

- 生效环境变量：source /etc/profile

3. 设置 HIVE 运行环境

```
cd $HIVE_HOME/conf && vim hive-env.sh
```

对应设置如下：

```
# 配置 Hadoop 安装路径
export HADOOP_HOME=/usr/hadoop/hadoop-2.7.7
# 配置 Hive 配置文件存放路径
export HIVE_CONF_DIR=/usr/hive/apache-hive-2.3.4-bin/conf
# 配置 Hive 运行资源库路径
export HIVE_AUX_JARS_PATH=/usr/hive/apache-hive-2.3.4-bin/lib
```

4. jline 版本冲突问题

由于客户端需要和 hadoop 通信，所以需要更改 Hadoop 中 jline 的版本，即保留一个高版本的 jline-jar 包。

```
cp $HIVE_HOME/lib/jline-2.12.jar $HADOOP_HOME/share/hadoop/yarn/lib/
```

5. 安装包复制到 slave1

```
scp -r /usr/hive root@slave1:/usr/
```

5.3 hive 元数据配置到 MySQL(服务器端 slave1)

1. 驱动拷贝

因为服务端需要和 Mysql 通信，所以服务端需要将 Mysql 的依赖包放在 Hive 的 lib 目录下。

mysql-connector-java 是 MySQL 的 JDBC 驱动包，用 JDBC 连接 MySQL 数据库时必须使用该 jar 包。

```
cp /usr/package277/mysql-connector-java-5.1.47-bin.jar /usr/hive/apache-hive-2.3.4-bin/lib/
```

2. Slave1 作为服务器端配置 hive

```
cd $HIVE_HOME/conf
```

hive 配置可以参考官方文档:

<https://cwiki.apache.org/confluence/display/Hive/AdminManual+Metastore+Administration>

```
vim hive-site.xml
```

```
<configuration>
<!-- Hive 产生的元数据存放位置-->
<property>
<name>hive.metastore.warehouse.dir</name>
<value>/user/hive_remote/warehouse</value>
</property>
<!-- 数据库连接 driver，即 MySQL 驱动-->
<property>
<name>javax.jdo.option.ConnectionDriverName</name>
<value>com.mysql.jdbc.Driver</value>
</property>
<!-- 数据库连接 JDBC 的 URL 地址-->
<property>
<name>javax.jdo.option.ConnectionURL</name>
<value>jdbc:mysql://slave2:3306/hive?createDatabaseIfNotExist=true&useSSL=false</value>
</property>
<!-- MySQL 数据库用户名-->
<property>
<name>javax.jdo.option.ConnectionUserName</name>
<value>root</value>
</property>
<!-- MySQL 数据库密码-->
<property>
<name>javax.jdo.option.ConnectionPassword</name>
<value>123456</value>
</property>
</configuration>
```

5.4 Master 作为客户端配置 hive

和 slave1 中配置方式类似，直接进入 conf，进行设置。

Hive 服务和 metastore 在不同的进程内，可能是不同的机器，该模式需要将 hive.metastore.local 设置为 false。将 hive.metastore.uris 设置 metastore 服务器 URL，端口设定为 9083。

```
cd $HIVE_HOME/conf
```

```
vim hive-site.xml
```

```
<configuration>
<!-- Hive 产生的元数据存放位置-->
<property>
<name>hive.metastore.warehouse.dir</name>
<value>/user/hive_remote/warehouse</value>
</property>
<!-- 使用本地服务连接 Hive,默认为 true-->
<property>
<name>hive.metastore.local</name>
<value>>false</value>
</property>
<!-- 连接服务器-->
<property>
<name>hive.metastore.uris</name>
<value>thrift://slave1:9083</value>
</property>
</configuration>
```

#Hive 服务和 metastore 在不同的进程内,可能是不同的机器,该模式需要将 hive.metastore.local 设置为 false, 将 hive.metastore.uris 设置为 metastore 服务器 URL。

#指向 metastore 服务的主机

5.5 成功启动 Hive

1.服务器端 slave1 初始化数据库

slave1 上:

```
schematool -dbType mysql -initSchema
```

2.启动 hive server 服务

设置成远程模式之后必须保证先启动 metastore 服务或者进程,之后才能进行开启客户端。

```
hive --service metastore & (注意空格, 后台运行)
```

3.客户端 master 启动 hive client

```
hive
```

创建数据库 hive

```
hive>create database hive;
```

6.安装 spark

6.1 安装 scala 环境

首先在根目录下建立工作路径/usr/scala。

```
mkdir -p /usr/scala
```

进入创建的 scala 工作路径。

```
cd /usr/scala
```

解压 java 安装包(master 上执行)。

```
tar -zxvf /usr/package277/scala-2.10.6.tgz -C /usr/scala/
```

- 修改环境变量: vim /etc/profile

添加内容如下:

```
export SCALA_HOME=/usr/scala/scala-2.10.6
export CLASSPATH=$SCALA_HOME/lib/
export PATH=$PATH:$SCALA_HOME/bin
export PATH SCALA_HOME CLASSPATH
```

- 生效环境变量: source /etc/profile

- 查看 java 版本: scala -version

在 master 中将 JDK 复制到 slave1 和 slave2 中。(在 master 执行)

```
scp -r /usr/scala root@slave1:/usr/
```

```
scp -r /usr/scala root@slave2:/usr/
```

注意: 此时需要去 slave1 和 slave2 上配置 scala 的环境变量, 并使环境变量生效。

6.2 安装 Spark

首先在根目录下建立工作路径/usr/spark。

```
mkdir -p /usr/spark
```

进入创建的 spark 工作路径。

```
cd /usr/spark
```

解压 java 安装包(master 上执行)。

```
tar -zxvf /usr/package277/spark-2.4.3-bin-hadoop2.7.tgz -C /usr/spark/
```

配置 spark 环境变量

命令: vim /etc/profile

在其中添加如下内容:

```
export SPARK_HOME=/usr/spark/spark-2.4.3-bin-hadoop2.7
export PATH=$SPARK_HOME/bin:$PATH
```

使环境变量生效: source /etc/profile

配置 spark-env.sh 文件, 并添加如下内容, 具体操作如下图所示:

```
cd /usr/spark/spark-2.4.3-bin-hadoop2.7/conf
```

```
vim spark-env.sh
```

```
export SPARK_MASTER_IP=master
export SCALA_HOME=/usr/scala/scala-2.10.6
export SPARK_WORKER_MEMORY=8g
export JAVA_HOME=/usr/java/jdk1.8.0_221
export HADOOP_HOME=/usr/hadoop/hadoop-2.7.7
export HADOOP_CONF_DIR=/usr/hadoop/hadoop-2.7.7/etc/hadoop
```

配置 spark 从节点, 修改 slaves 文件

```
vim slaves
```

```
slave1
slave2
```

接下来向所有子节点发送 spark 配置好的安装包, 具体操作如下:

```
scp -r /usr/spark root@slave1:/usr/
```

```
scp -r /usr/spark root@slave2:/usr/
```

修改 slave1 和 slave2 的环境变量, 此步骤和修改 master 中 spark 环境变量相同, 这里就不多介绍了, 最后记得是环境变量生效。这时我们的 spark 环境就安装成功了。

测试 spark 环境

因为我们安装的是 spark on yarn 模式, 所有接下来我们需要启动 hadoop 集群之后, 开启 spark 集群我们只需要在 master 节点上执行如下命令即可。

```
/usr/spark/spark-2.4.3-bin-hadoop2.7/sbin/start-all.sh
```

The image displays three terminal windows illustrating the Spark startup process across a master and two slave nodes.

Terminal 1 (Master): Shows the execution of `/usr/spark/spark-2.4.0-bin-hadoop2.7/sbin/start-all.sh`. The output includes logs for the Master, Worker, and NameNode processes. Red arrows point to the `spark Master进程` (Spark Master process) and the `开启spark` (Start Spark) command.

Terminal 2 (Slave1): Shows the output of `jps` on the slave1 node. The output lists the following processes: `2811 DataNode`, `2924 NodeManager`, `3052 Jps`, `3081 Worker`, `3130 Jps`, `2811 DataNode`, and `2924 NodeManager`. A red arrow points to the `3081 Worker` process, labeled `slave1上的spark Worker进程` (Spark Worker process on slave1).

Terminal 3 (Slave2): Shows the output of `jps` on the slave2 node. The output lists the following processes: `4821 DataNode`, `4934 NodeManager`, `5055 Jps`, `5091 Worker`, `5140 Jps`, `4821 DataNode`, and `4934 NodeManager`. A red arrow points to the `5091 Worker` process, labeled `slave2上的spark Worker进程` (Spark Worker process on slave2).

8、数据统计分析

1.格式化集群并开启集群

格式化：hadoop namenode -format

启动集群：start-all.sh

2.开启 mysql 服务

systemctl start mysqld.service

3.初始化 hive 元数据库，进去 hive 客户端

schematool -dbType mysql -initSchema

进入 hive 客户端：hive

创建数据库：create database hive;

项目一：新冠疫情数据分析

1.项目需求

自 2019 年底，湖北省武汉市监测发现不明原因肺炎病例，中国第一时间报告疫情，迅速采取行动，开展病因学和流行病学调查，阻断疫情蔓延。

SARS-CoV-2 是一种有着高扩散能力的病毒，通过飞沫、直接接触和被感染的物体传播，其潜伏时间为 1 到 14 天，并且也由无症状感染者传播。大多数感染者仅表现出轻度至中度的呼吸道症状，或根本不表现任何症状。只有 5-10% 的感染者显示出完全的严重呼吸综合征，称为冠状病毒病（COVID）-19，能够人传人，进而所引发的全球大流行疫情，是全球自第二次世界大战以来面临的最严峻危机。截至目前，全球已有 200 多个国家和地区累计报告超过 2.1375 亿确诊病例，导致超过 445 万名患者死亡。

在此基础上，大数据技术应用发挥极大作用。通过城市监测，接触者追踪，疫苗接种等，将我们的疫情信息进行传达。为政府正确决策、精准施策提供了科学依据。强化了政府对疫情物资生产、筹集、投放的科学管控手段。为医疗救治、“群防群控”，防止疫情蔓延采取有效措施提供了科学数据和手段。科学分析预测疫情现状、趋势，适时准确地根据疫情变化把握防疫重点。

本项目为新型冠状病毒（COVID-19）疫情状况的时间序列数据仓库，选手可以通过对疫情历史数据的分析研究，以更好的了解疫情与疫情的发展态势，为抗击疫情之决策提供数据支持。

2.数据说明

数据截止到 2021 年 9 月全球确诊、感染、治愈、死亡人数，地理信息包含地区、国家、省、市 4 级信息，时间信息为年与时分秒，按照每个城市每天更新一次。

	ssc.continentName	ssc.countryName	ssc.provinceName	123.province_confirm	123.province_suspect	123.province_cured	123.province_dead	updateTime	ssc.cityName	123.city_confirm	123.city_suspect	123.city_cured	123.city
1	亚洲	中国	湖北省	68,148	0	63,629	4,512	2020-11-26 09:34:00.0	孝感	3,518	0	3,389	
2	亚洲	中国	湖北省	68,148	0	63,629	4,512	2020-11-26 09:34:00.0	鄂州	1,394	0	1,335	
3	亚洲	中国	湖北省	68,148	0	63,629	4,512	2020-11-26 09:34:00.0	荆门	928	0	887	
4	亚洲	中国	湖北省	68,148	0	63,629	4,512	2020-11-26 09:34:00.0	荆州	1,580	0	1,528	
5	亚洲	中国	湖北省	68,148	0	63,629	4,512	2020-11-26 09:34:00.0	武汉	50,353	0	46,477	
6	亚洲	中国	湖北省	68,148	0	63,629	4,512	2020-11-26 09:34:00.0	黄冈	2,907	0	2,782	
7	亚洲	中国	湖北省	68,148	0	63,629	4,512	2020-11-26 09:34:00.0	宜昌	931	0	894	
8	亚洲	中国	湖北省	68,148	0	63,629	4,512	2020-11-26 09:34:00.0	随州	1,307	0	1,262	
9	亚洲	中国	湖北省	68,148	0	63,629	4,512	2020-11-26 09:34:00.0	黄石	1,015	0	976	
10	亚洲	中国	湖北省	68,148	0	63,629	4,512	2020-11-26 09:34:00.0	襄阳	1,175	0	1,135	

字段	说明
----	----

continentName	大洲、地区
countryName	国家
provinceName	省份、地区、直辖市
province_confirmed	省份累计确诊人数
province_suspected	省份疑似感染人数
province_cured	省份治愈人数
province_dead	省份死亡人数
cityName	城市名
city_confirmed	城市累计确诊人数
city_suspected	城市疑似感染人数
city_cured	城市治愈人数
city_dead	城市死亡人数
updateTime	数据更新时间

3.算法思路

统计各地区以及各省份每日新增确诊人数，疑似人数，治愈人数以及死亡人数。

采用 Hive 分层对数据进行清洗，转换，分析。

- 1) ods 层为数据原始层，将历史数据直接导入。
- 2) dwd 层对数据进行清洗转换。
- 3) dwm 层对指标维度进行整合，获取今天以及下一天数据。
- 4) dws 层 统计分析各个地区每日新增确诊，感染，治愈，死亡的增量情况。
- 5) app 层对维度进行上卷，分析各省每日指标的增长量。

4.解题步骤

数据操作

```
'''
```

```
# 删除第一行字段名
```

```
sed -i "1d" /root/covid/covid_area.csv
```

```
'''
```

查看第一行数据：head -n 1 /root/covid/covid_area.csv

1.创建 ods 层数据表

在**ods 层**是数据**原始层**，不做任何处理，只需将原始数据拉去过来即可，**ods 层**采用**内部表**（EXTERNAL 修饰）保证我们数据安全性。

```
##### 创建 ods 层数据库：covid_ods
```

```
```sql
```

```
create database covid_ods;
```

```
use covid_ods;
```

```
'''
```

```
创建疫情数据表：covid
```

```
```sql
```

```
CREATE TABLE IF NOT EXISTS covid_ods.covid (
  `containerName` STRING COMMENT '地区',
  `countryName` STRING COMMENT '国家',
  `provinceName` STRING COMMENT '省份',
  `province_confirm` STRING COMMENT '省份确诊人数',
  `province_suspect` STRING COMMENT '省份感染人数',
  `province_cured` STRING COMMENT '省份治愈人数',
  `province_dead` STRING COMMENT '省份死亡人数',
  `cityName` STRING COMMENT '城市名',
  `city_confirm` STRING COMMENT '城市确诊人数',
  `city_suspect` STRING COMMENT '城市感染人数',
  `city_cured` STRING COMMENT '城市治愈人数',
  `city_dead` STRING COMMENT '城市死亡人数',
  `updateTime` STRING COMMENT '数据更新时间'
)
comment '全球疫情数据源数据表'
ROW FORMAT DELIMITED FIELDS TERMINATED BY ',';
...
```

(2)加载数据;

```
``shell
load data local inpath '/root/covid/covid_area.csv' into table covid_ods.covid;
...
```

验证:

```
hdfs dfs -ls /user/hive/warehouse/covid_ods.db/covid
covid_area.csv
```

创建工作表: covid_time

hive 数据库 covid_ods.db 创建 covid_time 表, 提取有用数据, 过滤重复值, 只保留每天最后更新的数据;

- 1) 有用数据: 包括省份, 城市/地区, 地区确诊, 地区疑似感染, 地区治愈, 地区死亡;
- 2) 过滤重复值, 数据中有同一天获取的多次疫情信息, 根据时间只保留每天最后更新的数据;
- 3) 同时要求国家为中国, 省份不为中国, 过滤地区空值。

```
``sql
```

--创建数据表(过滤重复值, 提取有用数据(四个指标, 省份, 城市, 获取每天最后更新数据时间点))

```
CREATE TABLE IF NOT EXISTS covid_ods.covid_time (
  `provinceName` STRING COMMENT '省份',
  `cityName` STRING COMMENT '城市名',
  `city_confirm` STRING COMMENT '城市确诊人数',
  `city_suspect` STRING COMMENT '城市感染人数',
  `city_cured` STRING COMMENT '城市治愈人数',
  `city_dead` STRING COMMENT '城市死亡人数',

```

```

        `updateTime` STRING COMMENT '数据更新时间'
    )
comment '全球疫情数据源数据表'
ROW FORMAT DELIMITED FIELDS TERMINATED BY ',';

--插入数据（过滤出处理数据以及最后更新数据时间以及过滤出内陆数据）
insert into table covid_ods.covid_time
select provinceName,cityName,city_confirm,city_suspect,city_cured,city_dead,updateTime
from (
    select *, row_number() over (partition by cityName,substr(updateTime,1,10) order by city_confirm
desc,city_suspect desc, city_cured desc ,city_dead desc ) as num from covid_ods.covid
    )t where t.num=1
and t.provinceName!='中国'
and
    t.countryName='中国'
and t.cityName!='';

//测试查询数据（以莆田数据为例）
//select * from covid_ods.covid_time where cityName='莆田' order by updateTime desc;
``

```

验证

```

hdfs dfs -cat /user/hive/warehouse/covid_ods.db/covid_time/000000_0 | grep 2021-09-20
福建省,莆田,246,0,56,0,2021-09-20 19:14:20

```

2.创建 dwd 层数据表

在**dwd 层**采用**分区表**将数据按照**年/月维度**进行分区存放，以便在获取某月数据时可快速获取，提高获取效率。在本层我们将获取**想要的字段数据**，**对不规则数据做简单整理**。

```

##### 创建 ods 层数据库：covid_dwd
``sql
create database covid_dwd;
use covid_dwd;
``

```

根据当天时间获取昨天对应时间列（为后续获取昨天数据做准备）：province

根据需求可以看出，全国各个省份每日确诊，疑似，治愈，死亡增长量中需要统计的指标维度分别为：

****指标****：城市累计确诊，城市累计疑似，城市累计治愈，城市累计死亡。

****维度****：省份，城市，时间（更新时间 updateTime，以及时间的前一天 yesterday）。

将数据中的时间切割，获取年月日；并获取昨天时间。

过滤数据：清除台湾等数据,过滤重复数据

```

```sql
--创建表
create table if not exists covid_dwd.province(
 `provinceName` STRING COMMENT '省份',
 `cityName` STRING COMMENT '城市',
 `city_confirm` STRING COMMENT '城市确诊人数',
 `city_suspect` STRING COMMENT '城市感染人数',
 `city_cured` STRING COMMENT '城市治愈人数',
 `city_dead` STRING COMMENT '城市死亡人数',
 `updateTime` STRING COMMENT '更新时间',
 `yesterday` STRING COMMENT '昨天时间'
)
comment '全国疫情'
partitioned by (yearinfo String, monthinfo String)
row format delimited fields terminated by ','
stored as orc
tblproperties ('orc.compress'='SNAPPY');

...

```sql
--动态分区配置
set hive.exec.dynamic.partition=true;
set hive.exec.dynamic.partition.mode=nonstrict;
--hive 压缩
--set hive.exec.compress.intermediate=true;
--set hive.exec.compress.output=true;
--写入时压缩生效
--set hive.exec.orc.compression.strategy=COMPRESSION;

--插入数据
insert into table covid_dwd.province partition(yearinfo,monthinfo)
select
    provinceName,
    cityName,
    city_confirm,
    city_suspect,
    city_cured,
    city_dead,
    substr(updateTime,1,10) as updateTime,
    date_sub(updateTime,1) as yesterday,
    substr(`updateTime`,1,4) as yearinfo,
    substr(`updateTime`,6,2) as monthinfo

```

```
from
covid_ods.covid_time;
...
```

查询全部数据并写入本地文件:

```
hive -e "set hive.cli.print.header=true;select * from covid_dwd.province" | sed 's/\t/,/g' > province.csv
```

3.DWM 数据处理分析

统计**每天各个省份**中**指标的增长量**，因此需要去获取**前一天或者后一天**的数据，在本层将**当天数据和前一天的数据**进行汇总，通过**join**方式将数据**合并为一条数据**。对四个指标数据进行类型转换，转换为 int 类型（在 dws 层将参与运算）。

创建 DWM 层数据库: covid_dwd

```
``sql
create database covid_dwm;
use covid_dwm;
...
```

创建当日数据和后一天数据汇总数据表: two_day

```
``sql
# 创建省份分析表
#dwm
create table if not exists covid_dwm.two_day(
    `provinceName` STRING COMMENT '省份',
    `cityName` STRING COMMENT '城市',
    `city_confirm` Int COMMENT '城市确诊人数',
    `city_suspect` Int COMMENT '城市感染人数',
    `city_cured` Int COMMENT '城市治愈人数',
    `city_dead` Int COMMENT '城市死亡人数',
    `updateTime` STRING COMMENT '更新时间',
    `city_confirm_before` Int COMMENT '一天前城市确诊人数',
    `city_suspect_before` Int COMMENT '一天前城市感染人数',
    `city_cured_before` Int COMMENT '一天前城市治愈人数',
    `city_dead_before` Int COMMENT '一天前城市死亡人数',
    `yesterday` STRING COMMENT '昨天日期'
)
comment '全国疫情'
partitioned by (yearinfo String, monthinfo String)
row format delimited fields terminated by ','
stored as orc
tblproperties ('orc.compress'='SNAPPY');
```

```

#设置参数
--动态分区配置
set hive.exec.dynamic.partition=true;
set hive.exec.dynamic.partition.mode=nonstrict;

--hive 压缩
--set hive.exec.compress.intermediate=true;
--set hive.exec.compress.output=true;
--写入时压缩生效
--set hive.exec.orc.compression.strategy=COMPRESSION;

--Error during job, obtaining debugging information...
--Error: Java heap space namenode 空间不足
set hive.exec.mode.local.auto=true;
--//只要大于 1024, hive 默认分配的内存分大一倍, 也就是 2048M
set mapreduce.map.memory.mb=1025;
set mapreduce.reduce.memory.mb=1025;
set hive.exec.mode.local.auto.input.files.max=25;

--SET mapreduce.map.memory.mb;
--SET mapreduce.reduce.memory.mb;
--SET yarn.nodemanager.vmem-pmem-ratio;

--SET mapreduce.reduce.memory.mb = 8092;
--SET mapreduce.reduce.memory.mb = 8092;

# 按省份分组统计
insert into table covid_dwm.two_day partition(yearinfo,monthinfo)
select a.provinceName as provinceName,
       a.cityName as cityName,
       cast(a.city_confirm as Int)  city_confirm,
       cast(a.city_suspect as Int)  city_suspect,
       cast(a.city_cured as Int)    city_cured,
       cast(a.city_dead as Int)    city_dead,
       a.updateTime as updateTime,
       cast(b.city_confirm as Int)  city_confirm_before,
       cast(b.city_suspect as Int)  city_suspect_before,
       cast(b.city_cured as Int)    city_cured_before,
       cast(b.city_dead as Int)    city_dead_before,
       a.yesterday as yesterday,
       substr(a.updateTime,1,4) as yearinfo,
       substr(a.updateTime,6,2) as monthinfo
from

```

```

    (select provinceName,cityName,city_confirm,city_suspect,city_cured,city_dead,updateTime,yesterday,
yearinfo,monthinfo from covid_dwd.province) a
  join
    (select provinceName,cityName,city_confirm,city_suspect,city_cured,city_dead,updateTime from
covid_dwd.province) b
  on
    a.yesterday=b.updateTime
  and
    a.cityName=b.cityName
  and
    a.provinceName=b.provinceName;
'''

```

查询全部数据并写入本地文件:

```

hive -e "set hive.cli.print.header=true;select * from covid_dwm.two_day" | sed 's/\t/,/g' >
/root/covid/two_day.csv

```

4.创建 dws 层

在**dwd 层**已经**拿到前一天的数据**，在本层**计算各个地区的指标增量**，计算方式为：
****每日指标增量=前一天指标数据-今日指标数据****

创建 dws 层数据库:

```

```sql
create database covid_dws;
use covid_dws;
'''

```

##### 创建 dws 层数据表:

##### (1) 疫情数据表:

```

```sql
create table if not exists covid_dws.day(
  provinceName STRING comment '省份',
  cityName STRING comment '市区',
  new_city_confirm INT comment '日确诊增长人数',
  new_cit_suspect INT comment '日疑似增长人数',
  new_city_cured INT comment '日治愈增长人数',
  new_city_dead INT comment '日死亡增长人数',
  updateTime STRING comment '日期'
)
comment '各省疫情每天增长表'
partitioned by (yearinfo String, monthinfo String)
row format delimited fields terminated by ','
stored as orc

```

```
tblproperties ('orc.compress'='SNAPPY');
...

##### 业务数据插入疫情数据表
```sql
#设置
--动态分区配置
set hive.exec.dynamic.partition=true;
set hive.exec.dynamic.partition.mode=nonstrict;
set hive.exec.max.dynamic.partitions.pernode=100000;
set hive.exec.max.dynamic.partitions=100000;
set hive.exec.max.created.files=100000;
--hive 压缩
--set hive.exec.compress.intermediate=true;
--set hive.exec.compress.output=true;
--写入时压缩生效
--set hive.exec.orc.compression.strategy=COMPRESSION;

insert into table covid_dws.day partition(yearinfo,monthinfo)
select
provinceName,
cityName,
(city_confirm - city_confirm_before) as new_city_confirm,
(city_suspect-city_suspect_before) as new_cit_suspect,
(city_cured-city_cured_before) as new_city_cured,
(city_dead-city_dead_before) as new_city_dead,
updateTime,
substr(updateTime,1,4) as yearinfo,
substr(updateTime,6,2) as monthinfo
from covid_dwm.two_day;
...

```

将 covid\_dws.day 内容保存至云主机/root/covid/day.csv:

```
hive -e "set hive.cli.print.header=true;select * from covid_dws.day" | sed 's/\t/,/g' > /root/covid/day.csv
```

#### ### 4.APP 层

##### ##### 1. 需求说明

针对疫情数据，我们在 app 层再对维度进行上卷，分析维度为各个省份每日的指标增量情况统计。

##### ##### 2. 创建 app 层数据库:

```
...
create database covid_app;
```

---

```
use covid_app;
``
```

##### 3. 创建 app 层业务表:

```
``sql
create table if not exists covid_app.day_app(
 provinceName STRING comment '省份',
 cityName STRING comment '市区',
 new_city_confirm INT comment '日确诊增长人数',
 new_cit_suspect INT comment '日疑似增长人数',
 new_city_cured INT comment '日治愈增长人数',
 new_city_dead INT comment '日死亡增长人数',
 updateTime STRING comment '日期'
)
comment '各省疫情每天增长表'
partitioned by (yearinfo String, monthinfo String)
row format delimited fields terminated by ','
stored as orc
tblproperties ('orc.compress'='SNAPPY');
``
```

插入数据

```
``sql
#设置
--动态分区配置
set hive.exec.dynamic.partition=true;
set hive.exec.dynamic.partition.mode=nonstrict;
set hive.exec.max.dynamic.partitions.pernode=100000;
set hive.exec.max.dynamic.partitions=100000;
set hive.exec.max.created.files=100000;
--hive 压缩
--set hive.exec.compress.intermediate=true;
--set hive.exec.compress.output=true;
--写入时压缩生效
set hive.exec.orc.compression.strategy=COMPRESSION;

#插入数据
insert into table covid_app.day_app partition(yearinfo,monthinfo)
select
 provinceName,
 '-1' as cityName,
```



---

```
sum(new_city_confirm) as sum_new_confirm,
sum(new_cit_suspect) as sum_new_suspect,
sum(new_city_cured) as sum_new_cured,
sum(new_city_dead) as sum_new_dead,
updateTime,
substr(updateTime,1,4) as yearinfo,
substr(updateTime,6,2) as monthinfo
from covid_dws.day
group by provinceName,updateTime;
``
```

将 covid\_app.day\_app 内容保存至云主机/root/covid/day\_app.csv.

```
hive -e "set hive.cli.print.header=true;select * from covid_app.day_app" | sed 's/\t/,/g' >
/root/covid/day_app.csv
```

## 9、重点说明

- 1.本次环境版本为 Hadoop2.7.7，对应 eclips 插件存放于云主机 master:/usr/package277/中。
- 2.阿里云上云主机搭建的 Hadoop 集群，集群之间通过内网通信，本地 eclipse 开发工具需要使用域名进行访问。

Hadoop 配置文件是以内网 IP 作为机器间通信的 IP。在这种情况下,我们能够访问到 namenode 机器,namenode 会给我们数据所在机器的 IP 地址供我们访问数据传输服务，但是当写数据的时候，NameNode 和 DataNode 是通过内网通信的，返回的是 datanode 内网的 IP,我们无法根据该 IP 访问 datanode 服务器。将默认的通过 IP 访问，改为通过域名方式访问。

```
// 设置客户端访问 datanode 使用 hostname 来进行访问
conf.set("dfs.client.use.datanode.hostname", "true");
conf.set("fs.defaultFS","hdfs://master:9000");
```